

TCP

Transmission Control Protocol – RFC 793

Projetado para oferecer um fluxo de bytes fim-a-fim confiável. Adaptar dinamicamente às propriedades da inter-rede. Pode ser uma biblioteca, um processo de usuário ou parte do núcleo. Divide os dados do usuário em partes de até 64 kB (ou de 1.460 bytes para caber num quadro ethernet). Administra os timers e nº de sequência para controle de retransmissão e ordem dos pacotes.

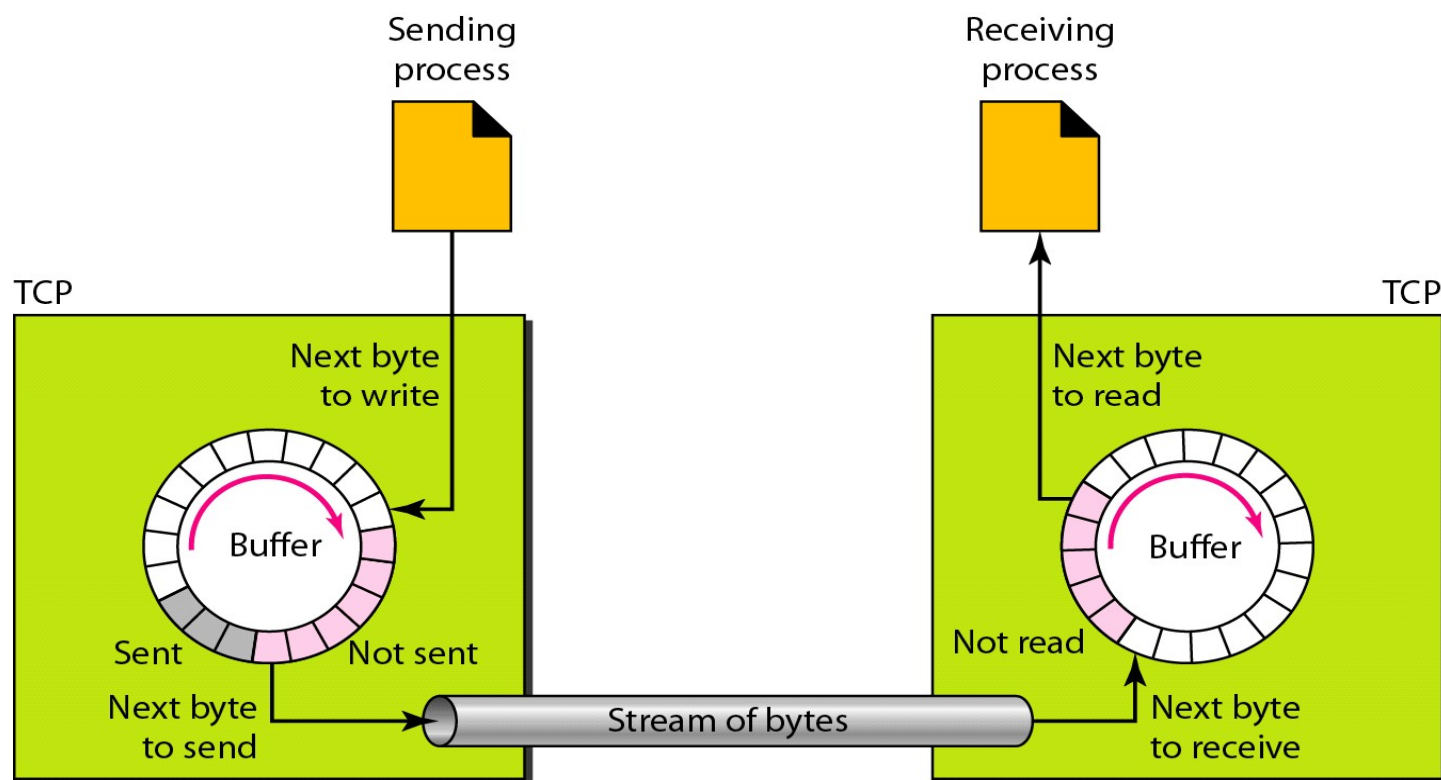
TCP

Serviços TCP.

- .Comunicação de Processo para Processo.
- .Serviço de Envio de Fluxo.
- .Buffers de envio e de recepção.
- .Segmentos.
- .Comunicação Full-duplex.
- .Serviços Orientado a conexão.
- .Serviço confiável.

Modelo de serviço do TCP

Criação de socket tanto no transmissor quanto no receptor;
Conexões explicitamente estabelecidas: IP + Porta;
Um soquete pode ser utilizado por mais de uma conexão e cada uma delas será identificada pelo par (soquete1, soquete2). Conexões full-duplex e ponto a ponto. Portas bem conhecidas (well-known ports) – serviços padrões:



Portas TCP bem conhecidas

<i>Port</i>	<i>Protocol</i>	<i>Description</i>
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call

O protocolo TCP

Cada byte tem uma conexão TCP possui seu próprio nº de seq. de 32 bits. Os dados são trocados em forma de segmentos.

Segmentos: cabeçalho fixo de 20 bytes + parte opcional + dados.

Restrições ao tamanho do segmento:

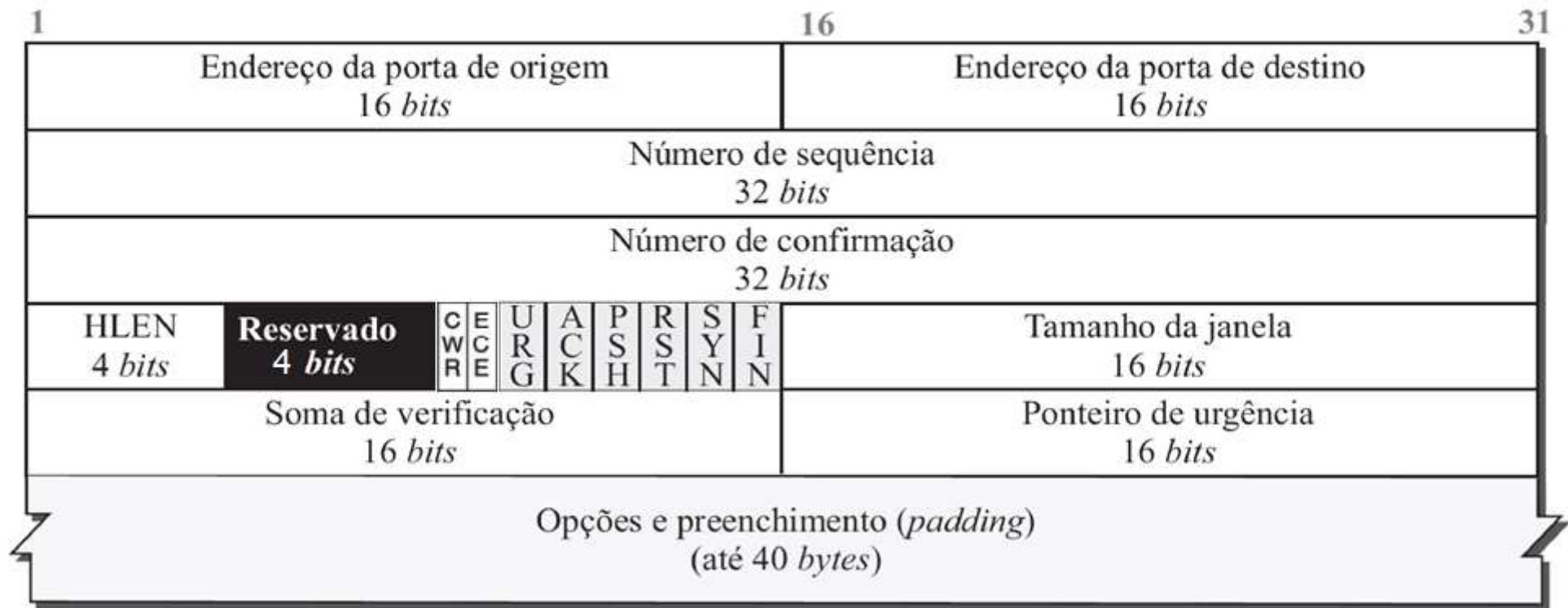
Cada segmento, incluindo o cabeçalho, deve caber dentro de um datagrama IP. Cada rede possui um MTU (Maximum Transfer Unit) que limita o tamanho do segmento (o tamanho da carga útil de uma MTU Ethernet é 1500 bytes).

Utiliza janela deslizante com timers. O receptor retorna um segmento (com ou sem dados, conforme o caso) com o nº de ack igual ao que ele deseja receber. Se o timer do transmissor expirar antes da chegada do ack, o segmento é retransmitido.

Cabeçalho do segmento TCP



(a) Segmento



(b) Cabeçalho

Cabeçalho TCP

TCP Header Length: nº de palavras de 32 bits que existem no cabeçalho, pois options tem tamanho variável.

URG – Urgent Pointer: Bit que em 1 força o TCP origem a parar de acumular dados no segmento e a enviá-lo imediatamente. No destino, interrompe qualquer processo em execução e processa a leitura do TPDU;

ACK: Em 1 indica que o acknowledgment number é um nº válido;

PSH – PUSH: Em 1 força o receptor a entregar os dados ao processo cliente imediatamente, sem bufferização.

RST – Reset: reinicializa um conexão com problemas.

SYN: Usado para estabelecer conexões:

Solicitação de conexão: SYN=1, ACK =0 (Connection Request).

Confirmação de conexão: SYN=1, ACK=1 (Connection Accept).

Cabeçalho TCP

FIN: Encerrar uma conexão. Indica que o transmissor não tem mais dados a transmitir, embora possa continuar indefinidamente a receber dados.

Tamanho da Janela: Utilizado para controle de fluxo:

Indica quantos bytes podem ser enviados a partir do ACK;

Window size=0,ACK=X: Confirma todos os dados recebidos até X-1 e informa que o buffer está cheio. Pede para aguardar. A liberação para novos envios é feita através de window size $\neq 0$ e Ack = X.

CWR: Sinaliza janela de congestionamento reduzida do transmissor TCP para o receptor TCP, de modo que ele saiba que o transmissor diminuiu a velocidade e pode parar de enviar uma mensagem ECN-Echo.

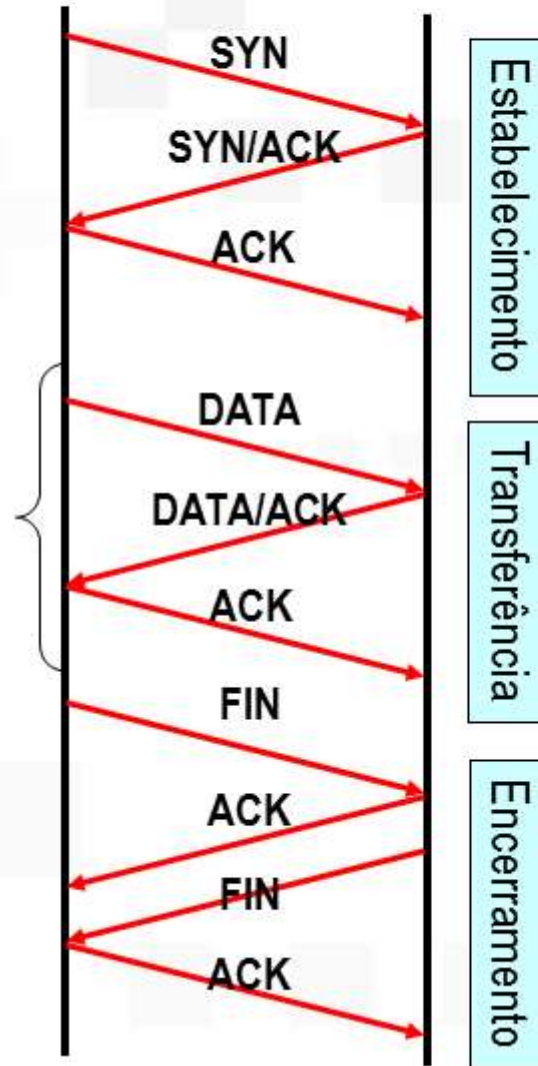
ECE: Sinaliza uma mensagem ECN-Echo a um transmissor para solicitar uma redução de velocidade quando o receptor TCP receber uma indicação de congestionamento de rede.

Opções: Opções adicionais, como estipulação do tamanho da carga útil.

Conexão TCP

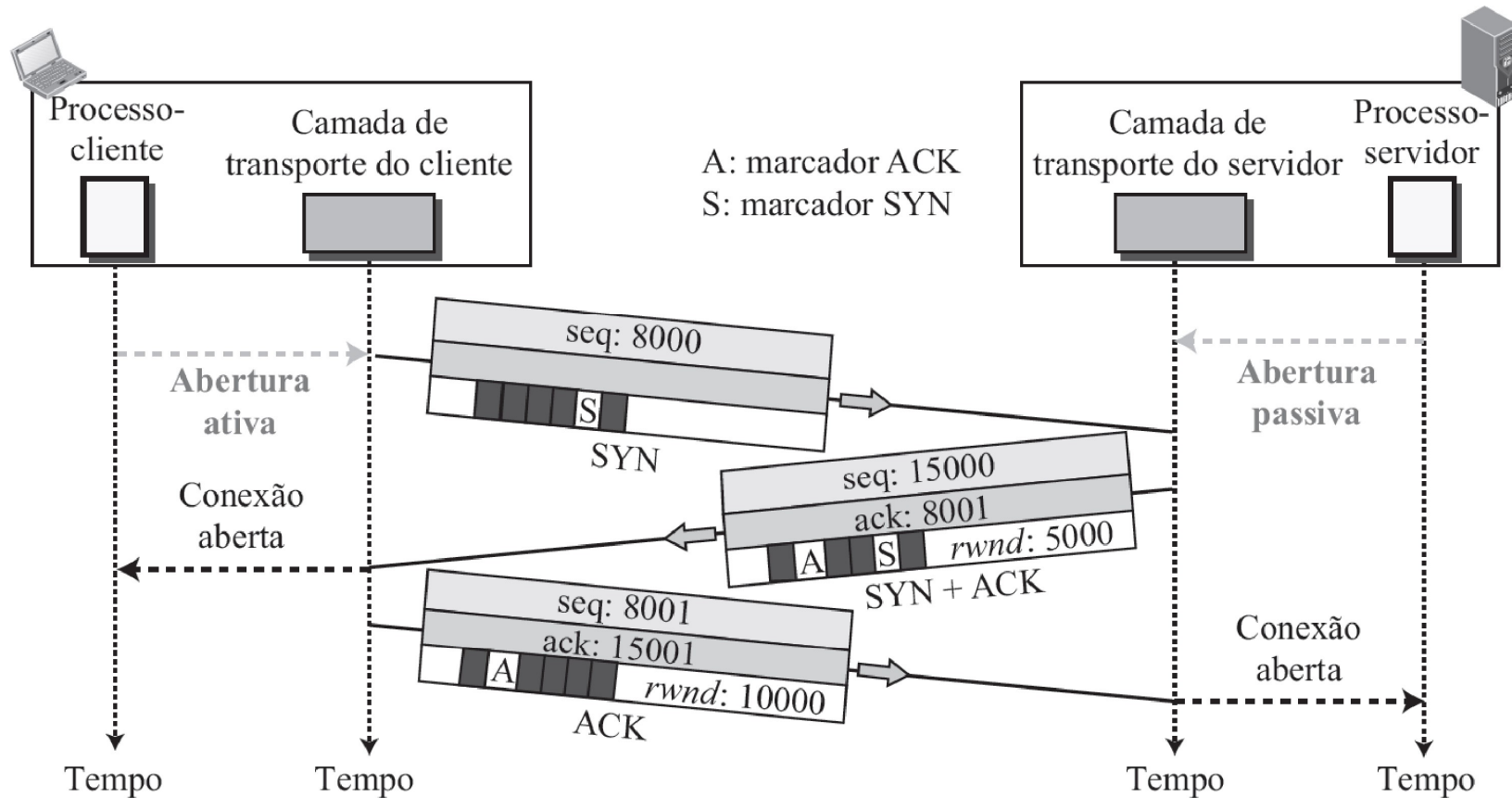


Campos *sequence number*, *acknowledgement number*, *window* e *urgent pointer* são usados nos mecanismos de controle de erro e de fluxo (ordenamento).



Estabelecimento da conexão TCP

three-way handshake



O diagrama ilustra o funcionamento de uma conexão TCP entre um cliente e um servidor. No topo, há duas caixas representando os sistemas: o cliente à esquerda e o servidor à direita. Cada caixa contém um ícone de processador e uma caixa rotulada 'Camada de transporte'. Abaixo das caixas, há duas linhas verticais tracejadas representando o tempo.

Entre as linhas de tempo, há uma barra horizontal escura no topo rotulada 'Estabelecimento da conexão' e outra no fundo rotulada 'Finalização da conexão'.

Os dados são transmitidos em segmentos TCP, representados por retângulos inclinados. Cada segmento contém os seguintes campos:

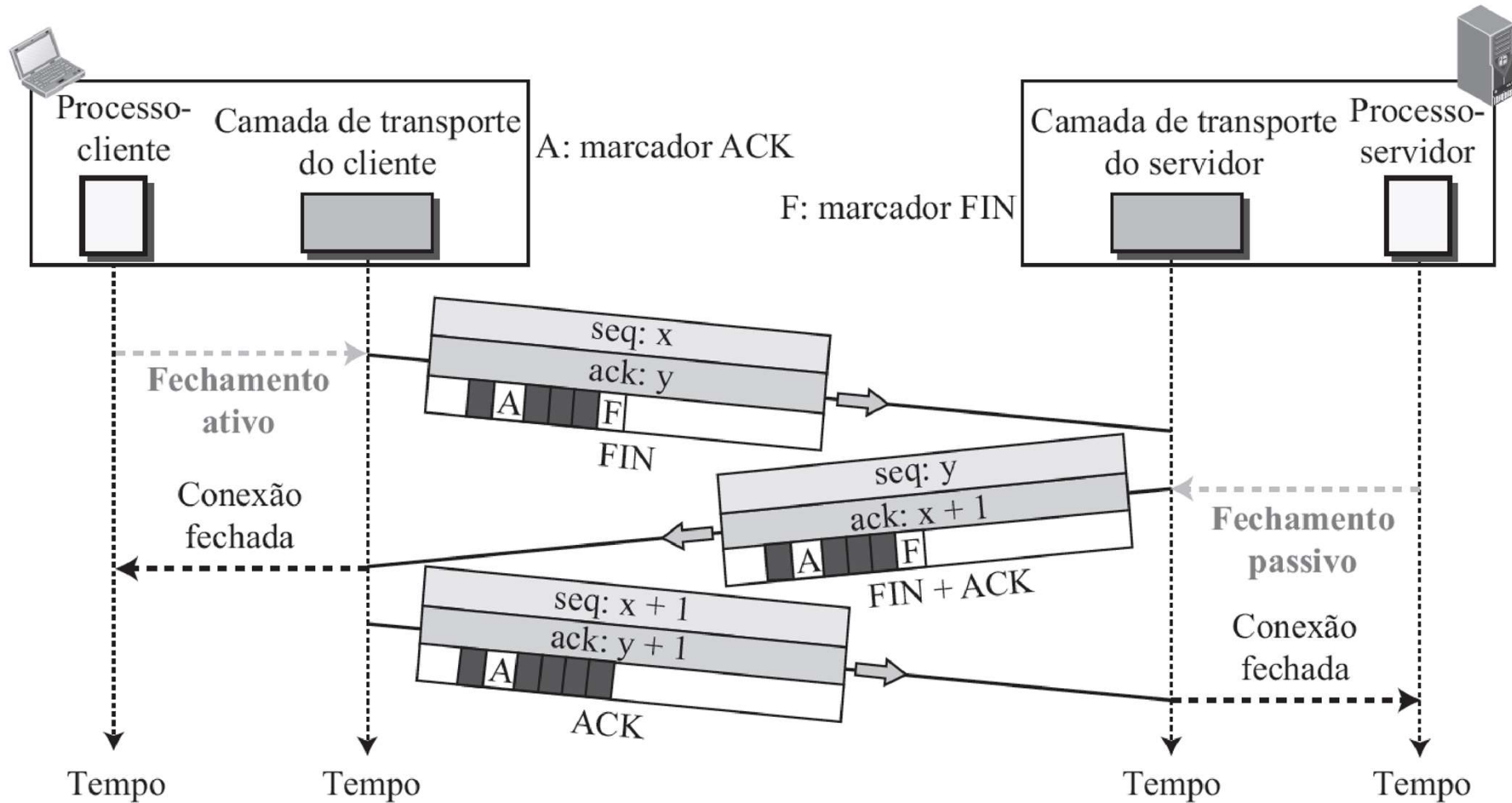
- seq:** número de sequência
- ack:** número de confirmação
- bytes:** intervalo de bytes transmitidos
- rwnd:** janela de recepção (presente nos segmentos de confirmação)
- Controle:** bits de controle como 'P' (PSH) e 'A' (ACK)

A sequência de eventos é a seguinte:

- O cliente envia uma requisição (seta tracejada) para o servidor.
- O servidor recebe a requisição (seta tracejada) e envia uma requisição de volta ao cliente (seta tracejada).
- O cliente envia um segmento de dados (seq: 8001, ack: 15001, bytes: 8001-9000) para o servidor.
- O servidor recebe o segmento (seta tracejada) e envia um segmento de dados (seq: 9001, ack: 15001, bytes: 9001-10000) para o cliente.
- O cliente recebe o segmento (seta tracejada) e envia um segmento de dados (seq: 10001, ack: 17001, bytes: 10001-11000) para o servidor.
- O servidor recebe o segmento (seta tracejada) e envia um segmento de confirmação (seq: 15001, ack: 10001, rwnd: 3000, bytes: 15001-17000) para o cliente.
- O cliente recebe o segmento (seta tracejada) e envia um segmento de confirmação (seq: 10001, ack: 17001, rwnd: 10000, bytes: 10001-11000) para o servidor.
- O servidor recebe o segmento (seta tracejada) e envia uma requisição de volta ao cliente (seta tracejada).
- O cliente recebe a requisição (seta tracejada) e envia uma requisição de volta ao servidor (seta tracejada).

As setas sólidas representam a transmissão de dados, enquanto as setas tracejadas representam as requisições e respostas de controle.

Encerramento da conexão TCP



Semi fechamento da conexão TCP

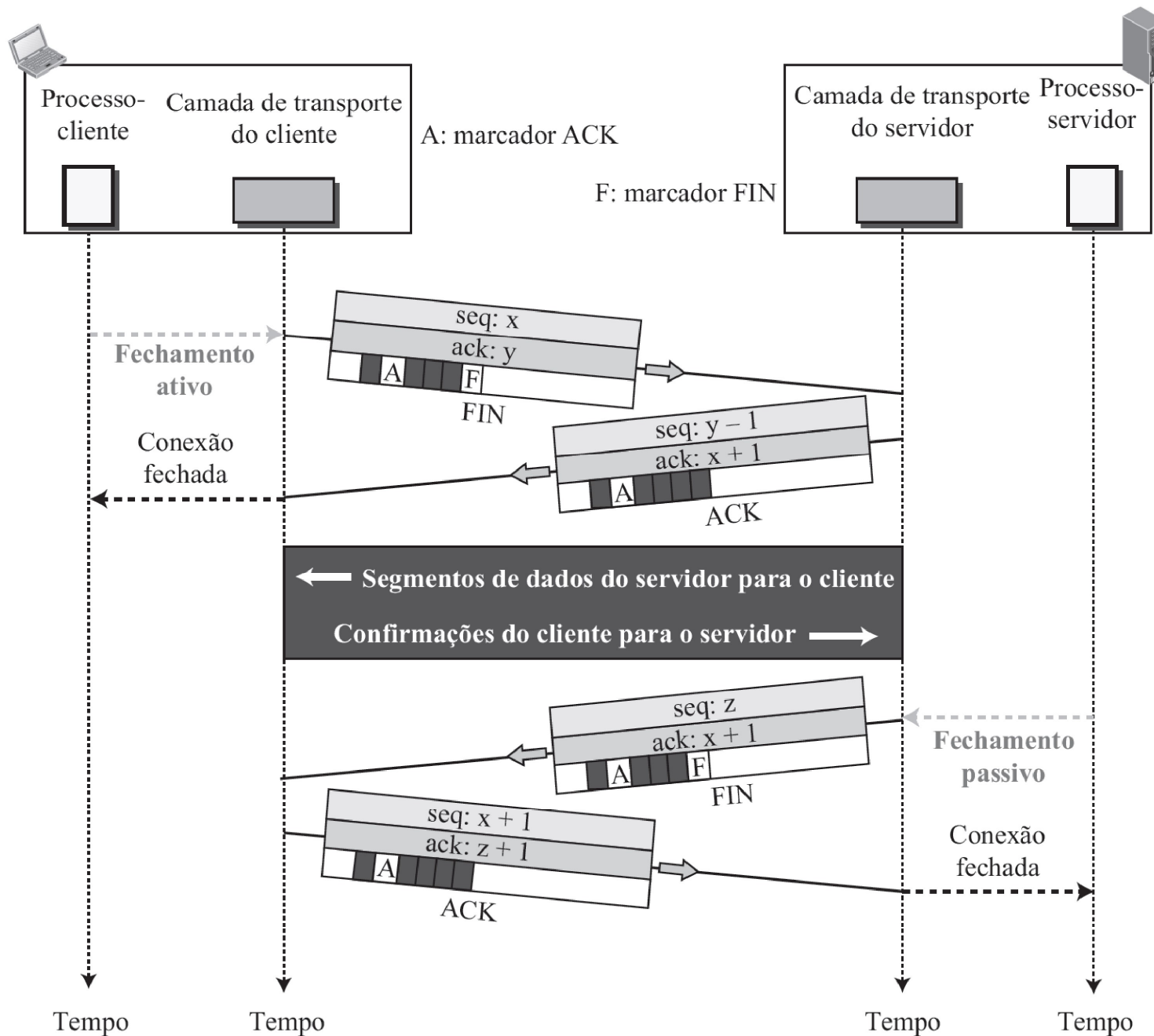
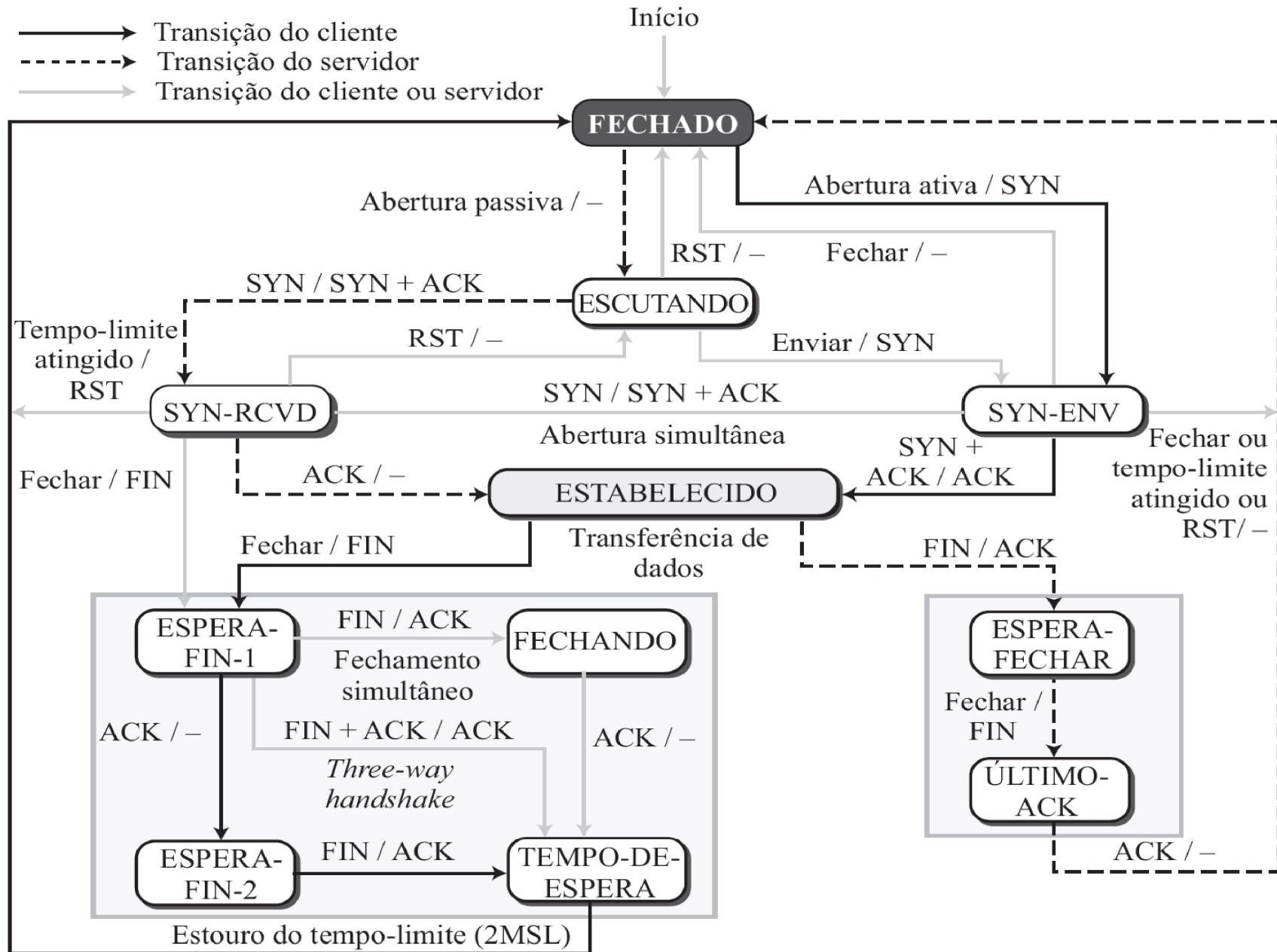


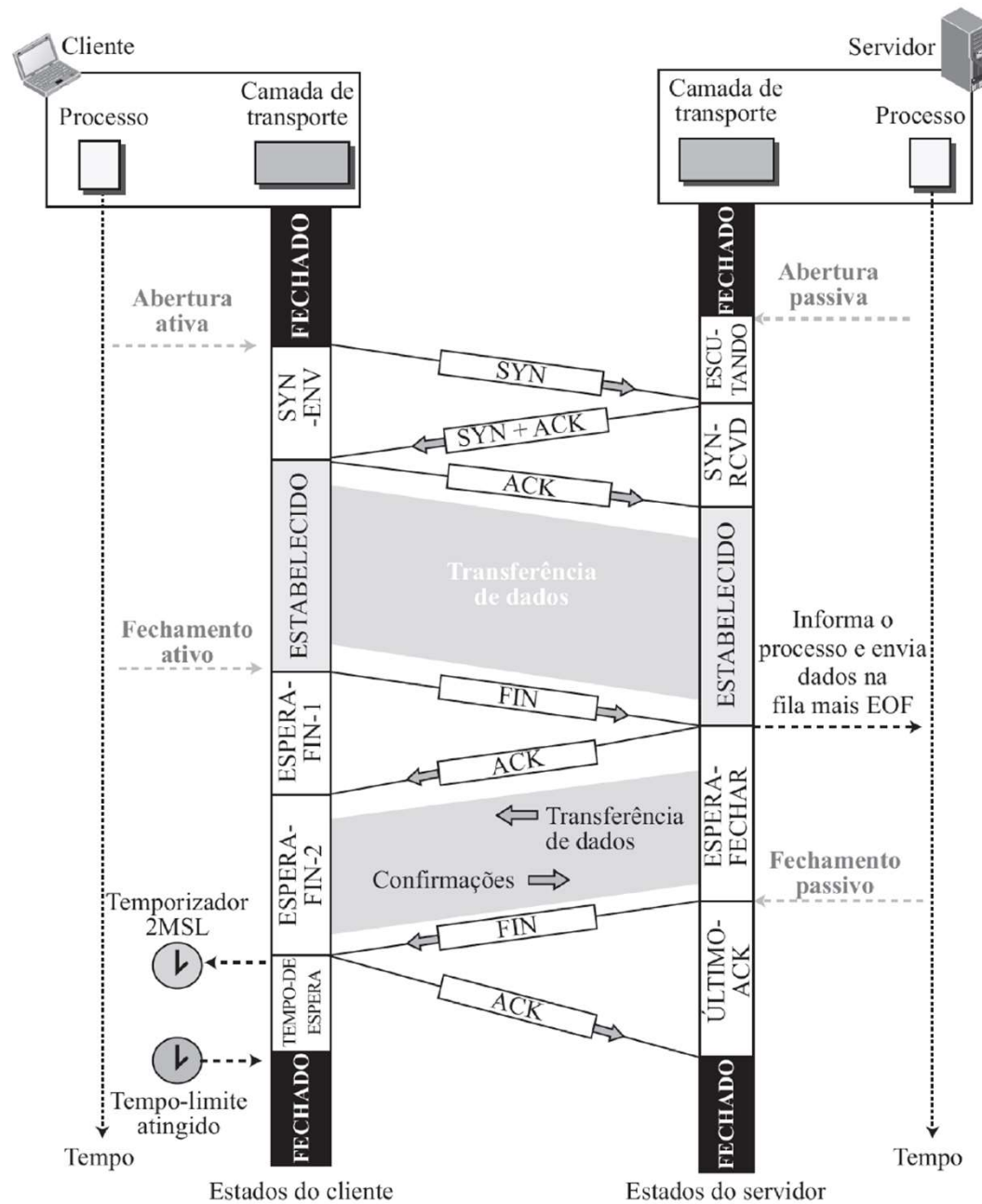
Diagrama de Estados do TCP



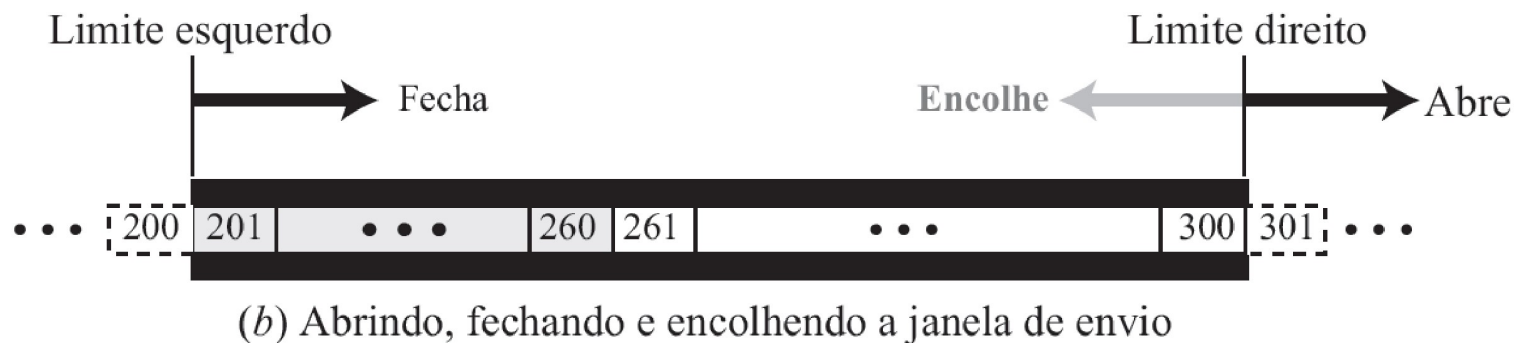
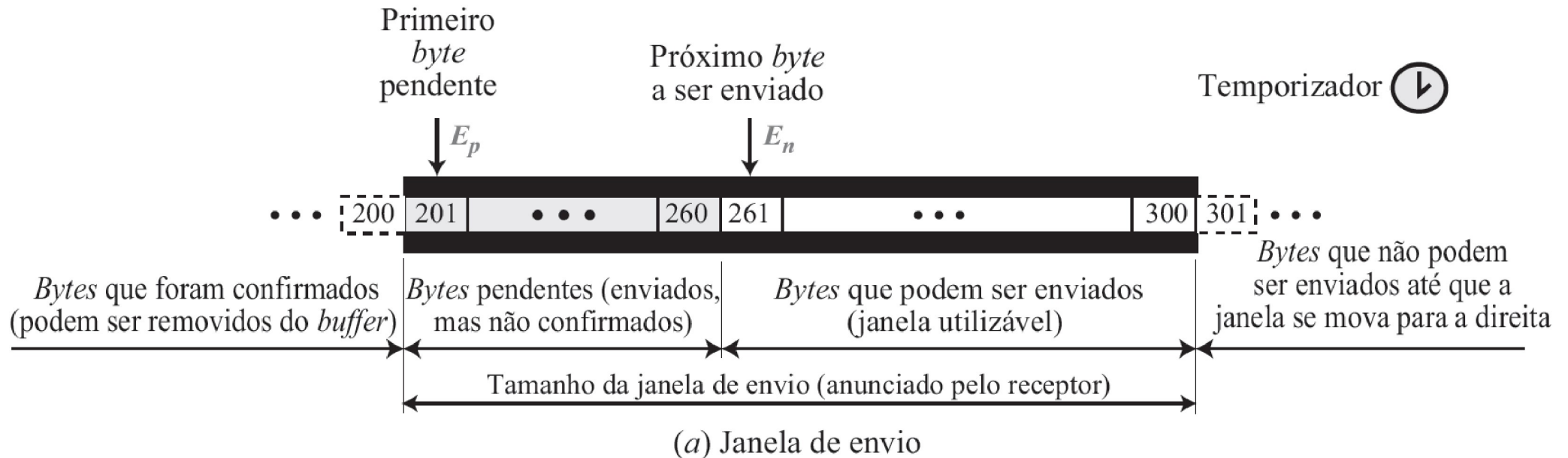
Estados do TCP

Estado	Descrição
FECHADO	Não há uma conexão
ESCUTANDO	Abertura passiva recebida; esperando por SYN
SYN-ENV	SYN enviado; esperando por ACK
SYN-RCVD	SYN+ACK enviado; esperando por ACK
ESTABELECIDO	Conexão estabelecida; transferência de dados em andamento
ESPERA-FIN-1	Primeiro FIN enviado; esperando por ACK
ESPERA-FIN-2	ACK para o primeiro FIN recebido; esperando por segundo FIN
ESPERA-FECHAR	Primeiro FIN recebido, ACK enviado; esperando pelo fechamento da aplicação
TEMPO-DE-ESPERA	Segundo FIN recebido, ACK enviado; esperando pela expiração do temporizador 2MSL
ÚLTIMO-ACK	Segundo FIN enviado; esperando por ACK
FECHANDO	Ambos os lados decidem fechar simultaneamente

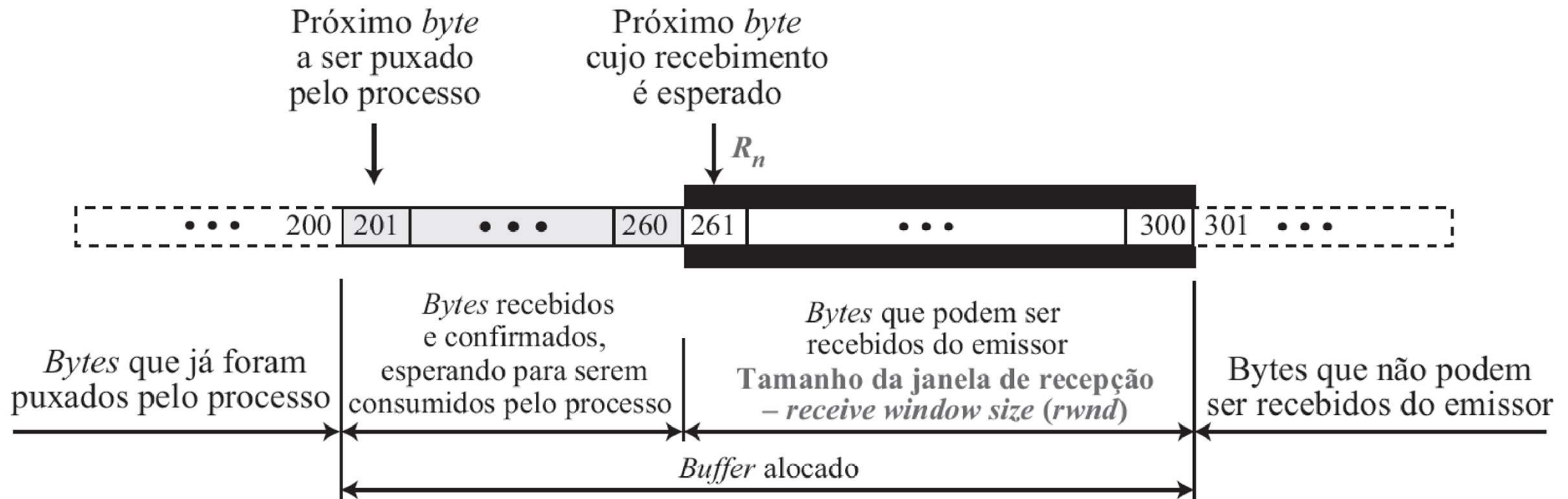
Cenário comum TCP



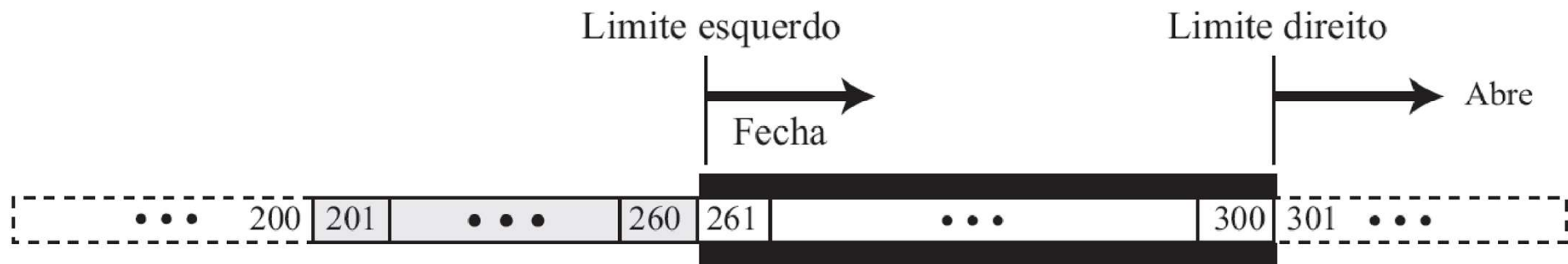
Janela de envio no TCP



Janela de recepção no TCP



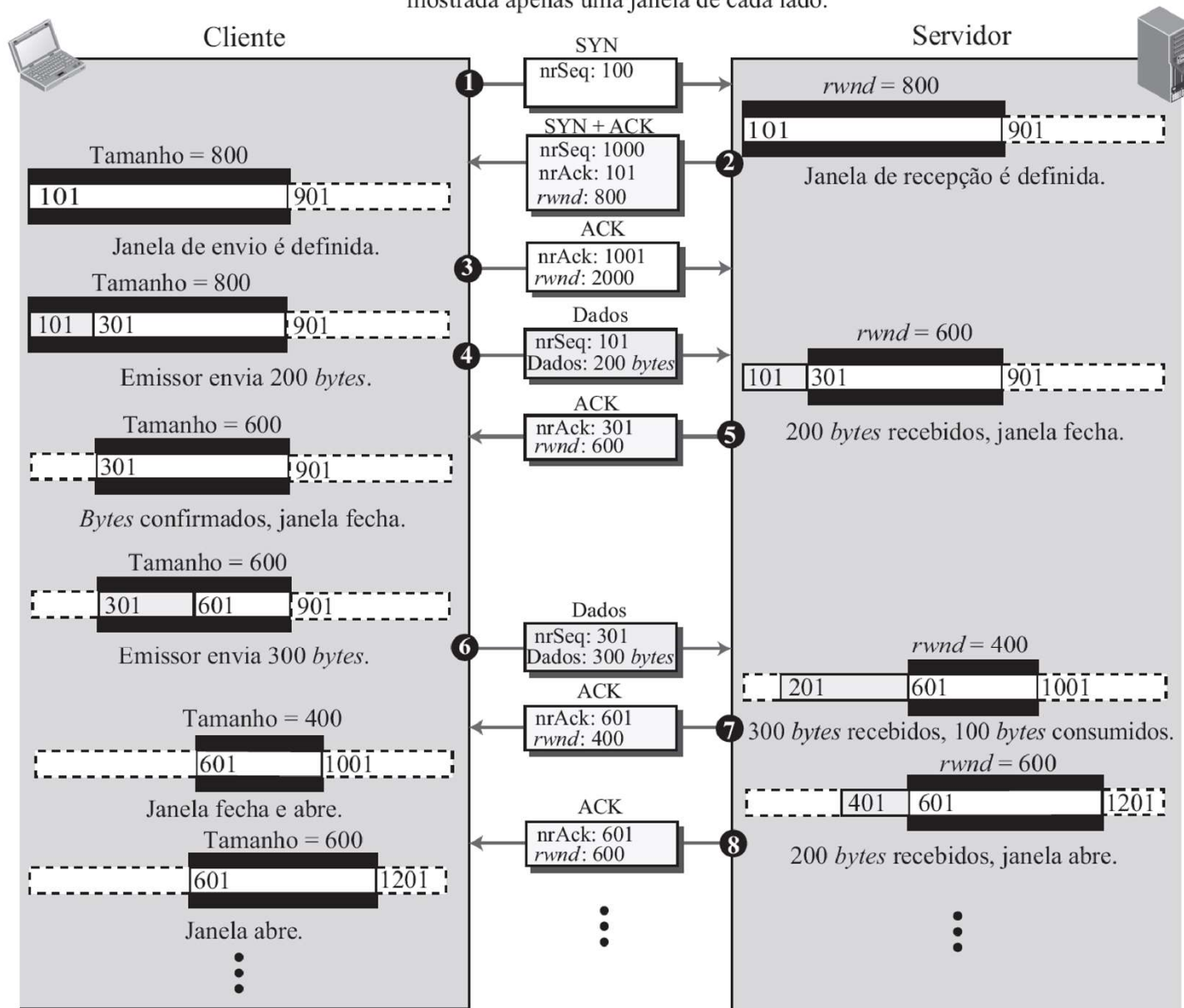
(a) Janela de recepção e *buffer* alocado



(b) Abrindo e fechando a janela de recepção

Cenário de controle de fluxo no TCP

Nota: Consideramos uma comunicação puramente unidirecional do cliente para o servidor. Consequentemente, é mostrada apenas uma janela de cada lado.



Confirmações

Confirmação Cumulativa (ACK)

Confirmação Seletiva (SACK)

Confirmações Cumulativas

Confirmação Cumulativa (ACK) O TCP foi originalmente projetado para confirmar a recepção de segmentos cumulativamente. O receptor anuncia o próximo byte que espera receber, ignorando todos os segmentos recebidos e armazenados fora de ordem. Isto é conhecido como confirmação cumulativa positiva, ou ACK. O campo de ACK de 32 bits no cabeçalho TCP é usado para confirmações cumulativas, e seu valor é válido somente quando o bit do marcador ACK vale 1.

Confirmações Seletivas

Confirmação Seletiva (SACK): O SACK informa quando um bloco de bytes está fora de ordem e também quando um bloco de bytes foi duplicado, ou seja, foi recebido mais de uma vez. No entanto, como não existe qualquer campo específico disponível no cabeçalho TCP para adicionar esse tipo de informação, o pacote de SACK é implementado como uma opção no fim do cabeçalho TCP. Um SACK não substitui um ACK, mas transmite informações adicionais ao remetente.

Regras de Confirmações

Regra 1: Quando a extremidade A envia um segmento de dados para a extremidade B, ela deve incluir (usando o mecanismo de carona, ou piggybacking) uma confirmação que informe o próximo número de sequência que espera receber. Essa regra diminui o número de segmentos necessários e, portanto, reduz o tráfego.

Regras de Confirmações

Regra 2: Quando o receptor não tem dados para enviar e recebe um segmento fora da ordem (com um número de sequência esperado), sendo que o segmento anterior já foi confirmado, o receptor atrasa o envio de um segmento de ACK até que outro segmento chegue ou até que um tempo-limite (normalmente de 500 ms) seja atingido. Em outras palavras, o receptor precisa atrasar o envio de um segmento de ACK se houver apenas um segmento pendente que esteja fora da ordem. Essa regra reduz o número de segmentos de ACK.

Regras de Confirmações

Regra 3: Quando um segmento chega com um número de sequência que é esperado pelo receptor e o segmento anterior recebido em ordem não foi confirmado, o receptor envia imediatamente um segmento de ACK. Em outras palavras, não deve haver mais do que dois segmentos sem confirmação que estejam em ordem em qualquer instante. Isto impede a retransmissão desnecessária de segmentos, algo que poderia criar congestionamento na rede.

Regras de Confirmações

Regra 4: Quando um segmento chega com um número de sequência fora de ordem que seja maior do que o esperado, o receptor imediatamente envia um ACK anunciando o número de sequência do próximo segmento esperado. Isto leva à retransmissão rápida dos segmentos faltantes.

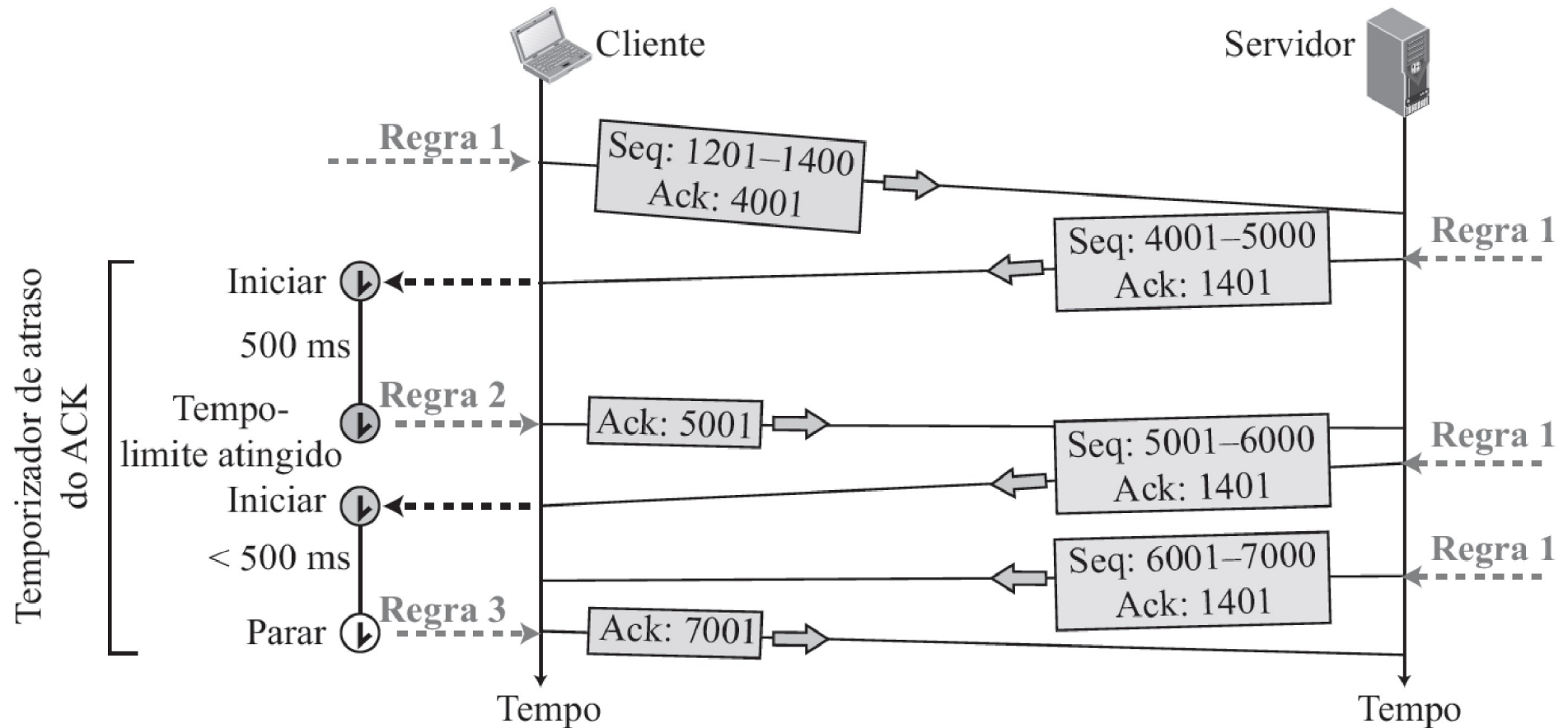
Regra 5: Quando um segmento que estava faltando chega ao receptor, ele envia um segmento de ACK para anunciar o próximo número de sequência esperado. Isto informa ao receptor que o segmento faltante foi recebido.

Regras de Confirmações

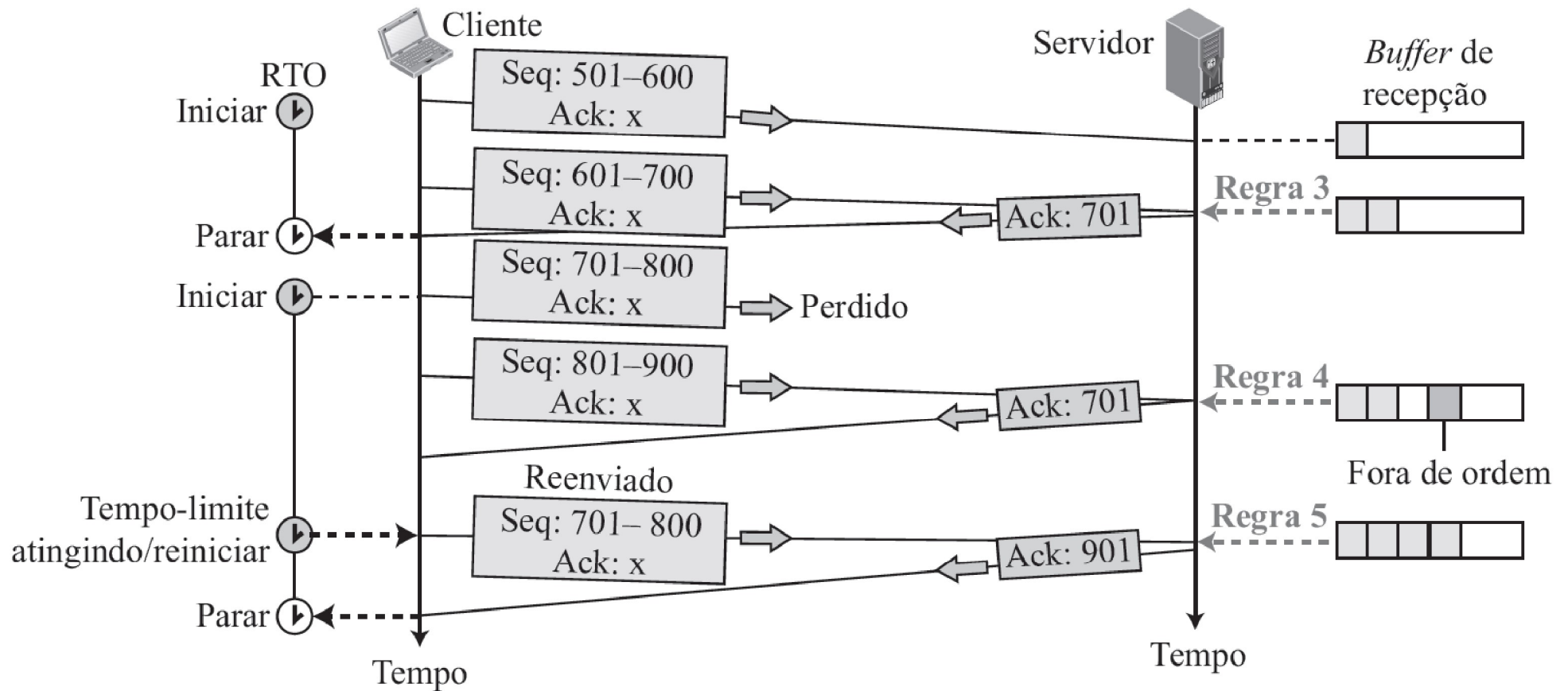
Regra 6: Se um segmento duplicado chegar ao receptor, ele descarta o segmento, porém envia imediatamente uma confirmação indicando o próximo segmento esperado, considerando a ordem correta. Isto resolve alguns problemas quando um segmento de ACK for perdido

Transmissão com TCP

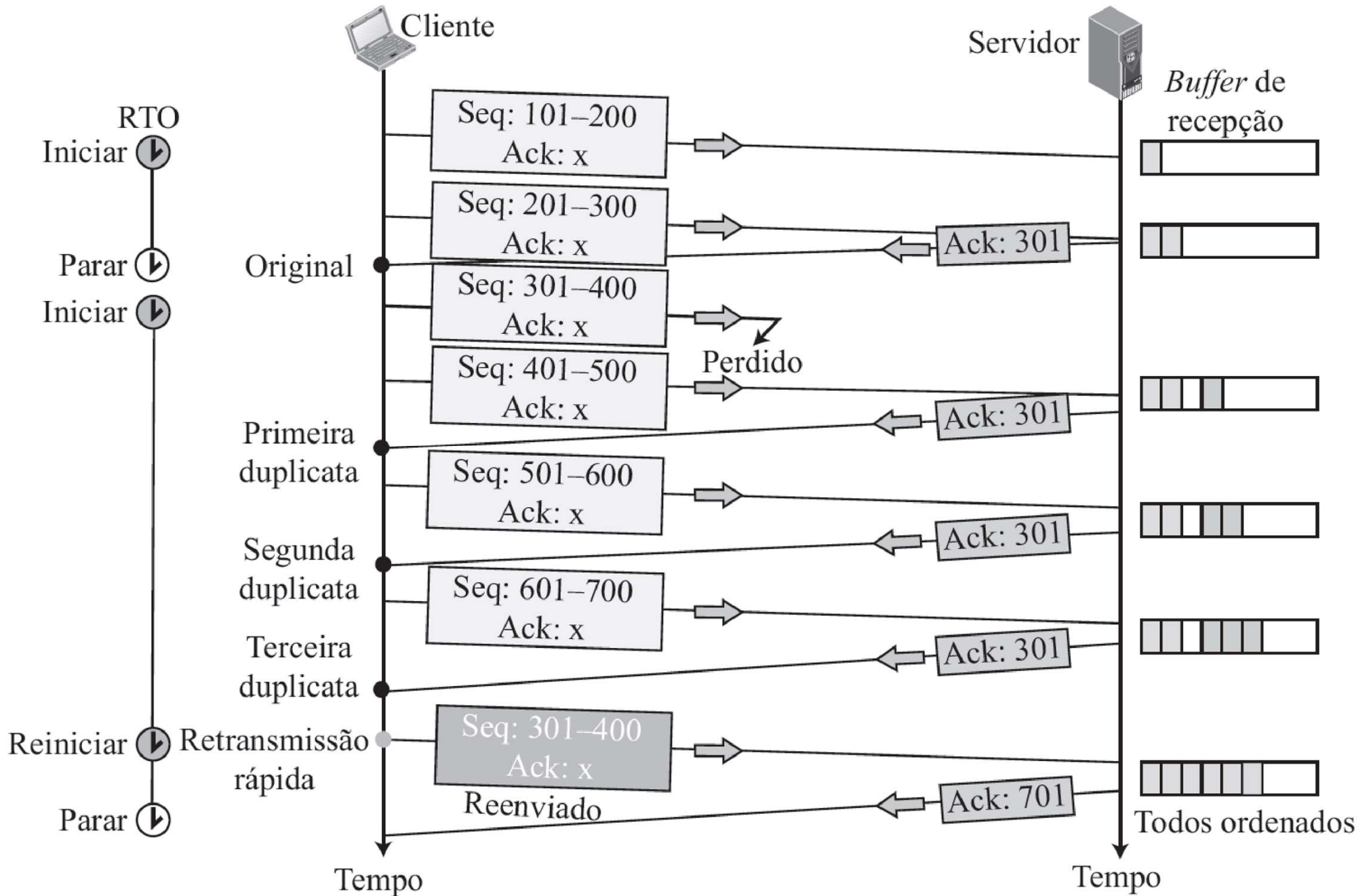
operação normal



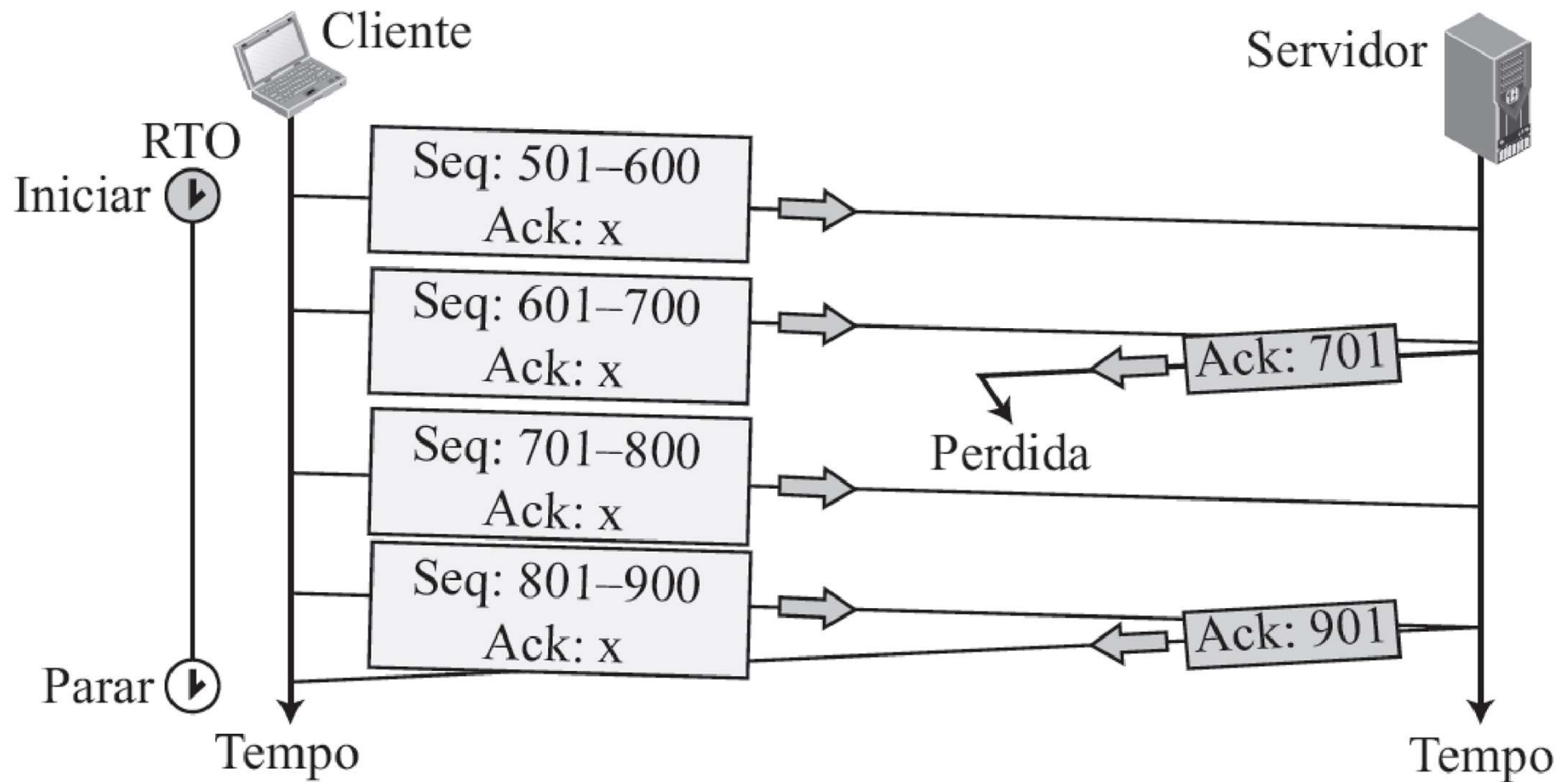
Perda de segmento



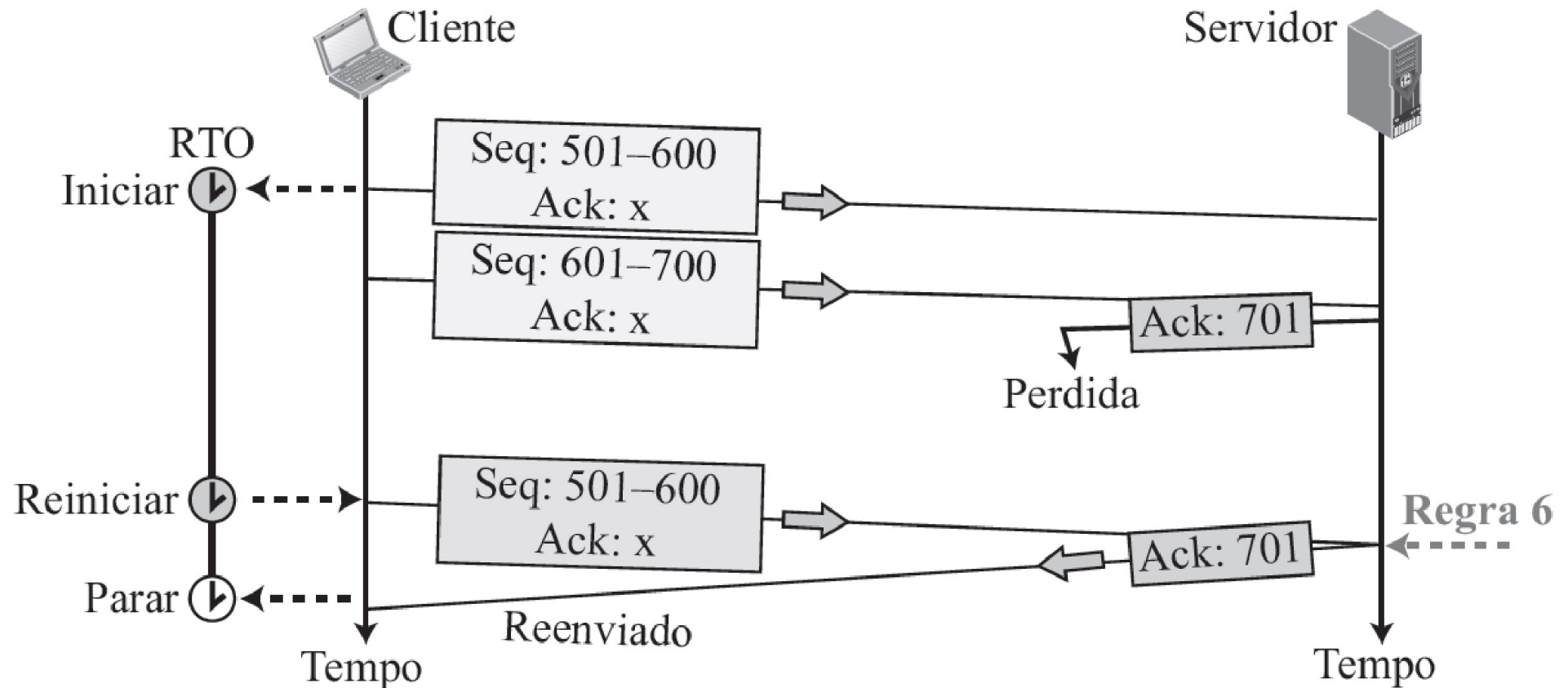
Retransmissão rápida



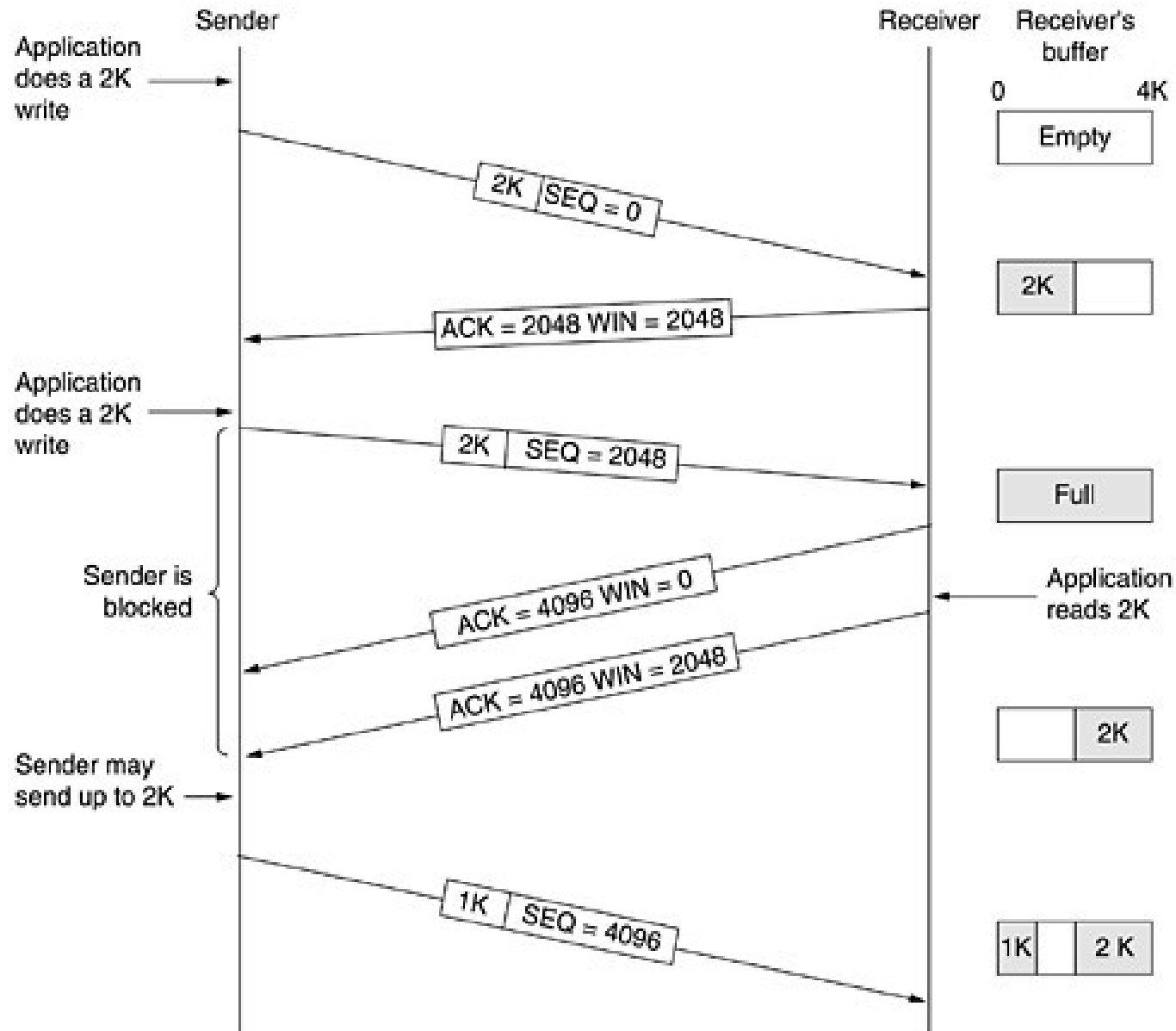
Confirmação rápida



Correção de confirmação perdido



Gerenciamento de Janelas no TCP



Gerenciamento de janelas

O TCP envia junto com todo ACK, o tamanho do buffer disponível para receber dados (Window Size)

Se Windows Size=0, o receptor com buffer lotado Normalmente o buffer de saída do TCP aguarda alguns ms antes de ser transmitido para que possa ser enviado mais cheio reduzindo overhead do protocolo.

Uma aplicação que consome 1 byte de cada vez do buffer está usando a pilha TCP de forma ineficiente porque assim que consumir 1 byte, o TCP enviará um Window Size de 1, permitindo o envio de mais um byte.

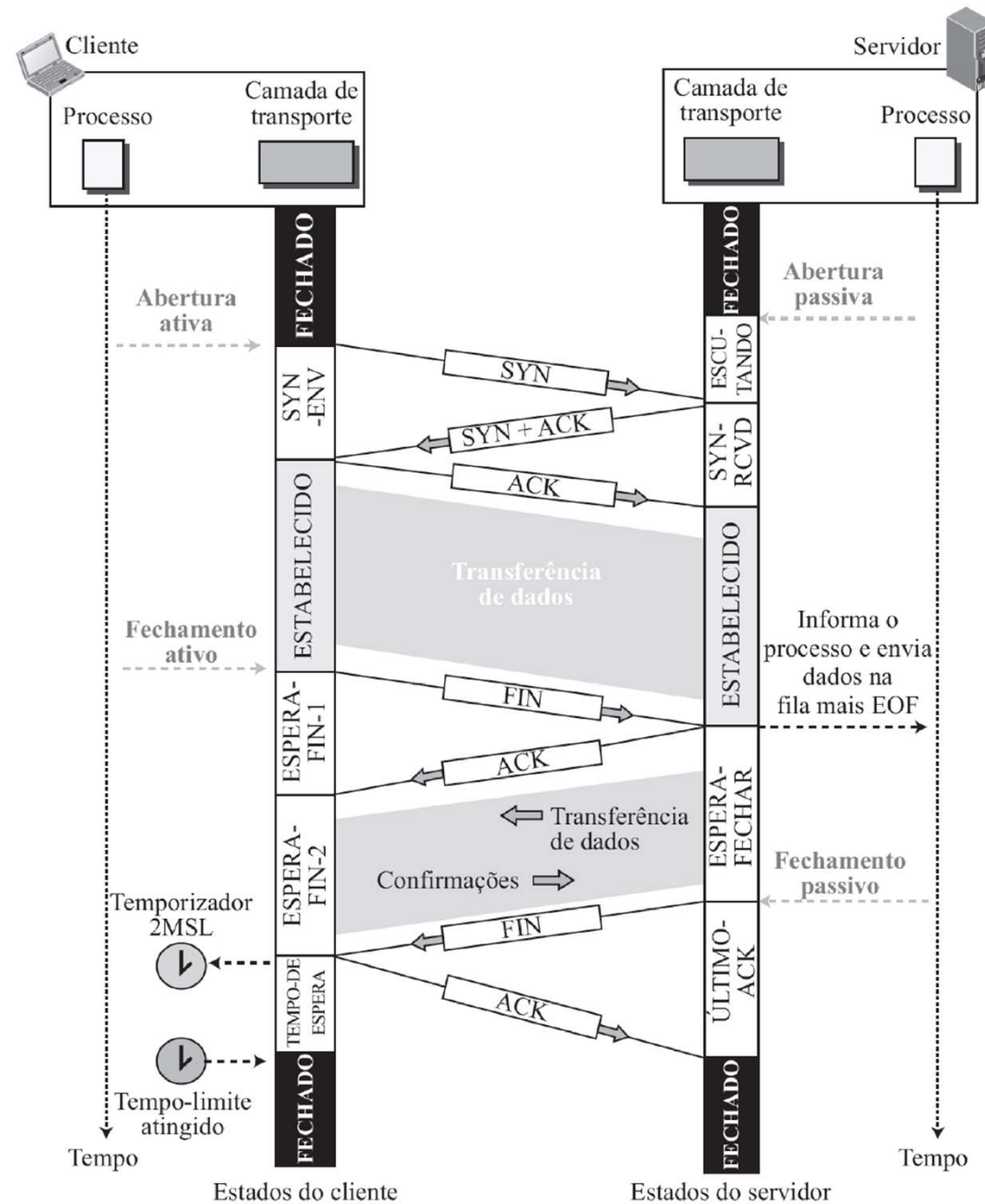
Algoritmo de Nagle:

O transmissor recebe 1 byte por vez da camada de aplicação, o 1o byte é transmitido assim que chega e os demais são armazenados até que chegue a confirmação do 1o byte, neste momento todos os bytes armazenados são enviados em um único pacote e o algoritmo retorna ao estado inicial.

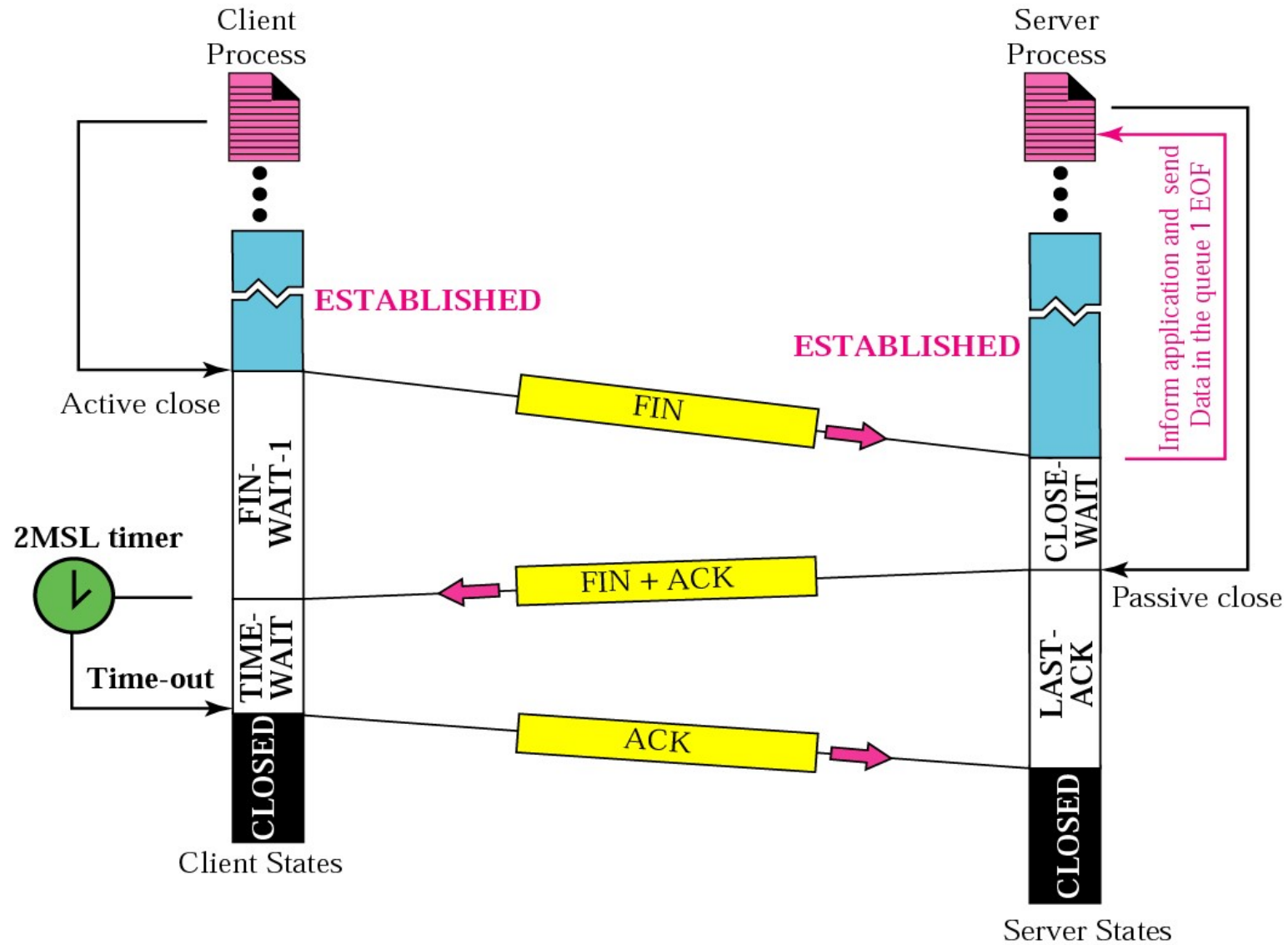
Em alguns casos não é interessante usar este algoritmo, por ex., quando estão sendo transmitidas informações sobre movimentação do mouse.

Para evitar problemas quando a janela começa a esvaziar o emissor deve aguardar um certo instante até a janela estar razoavelmente vazia.

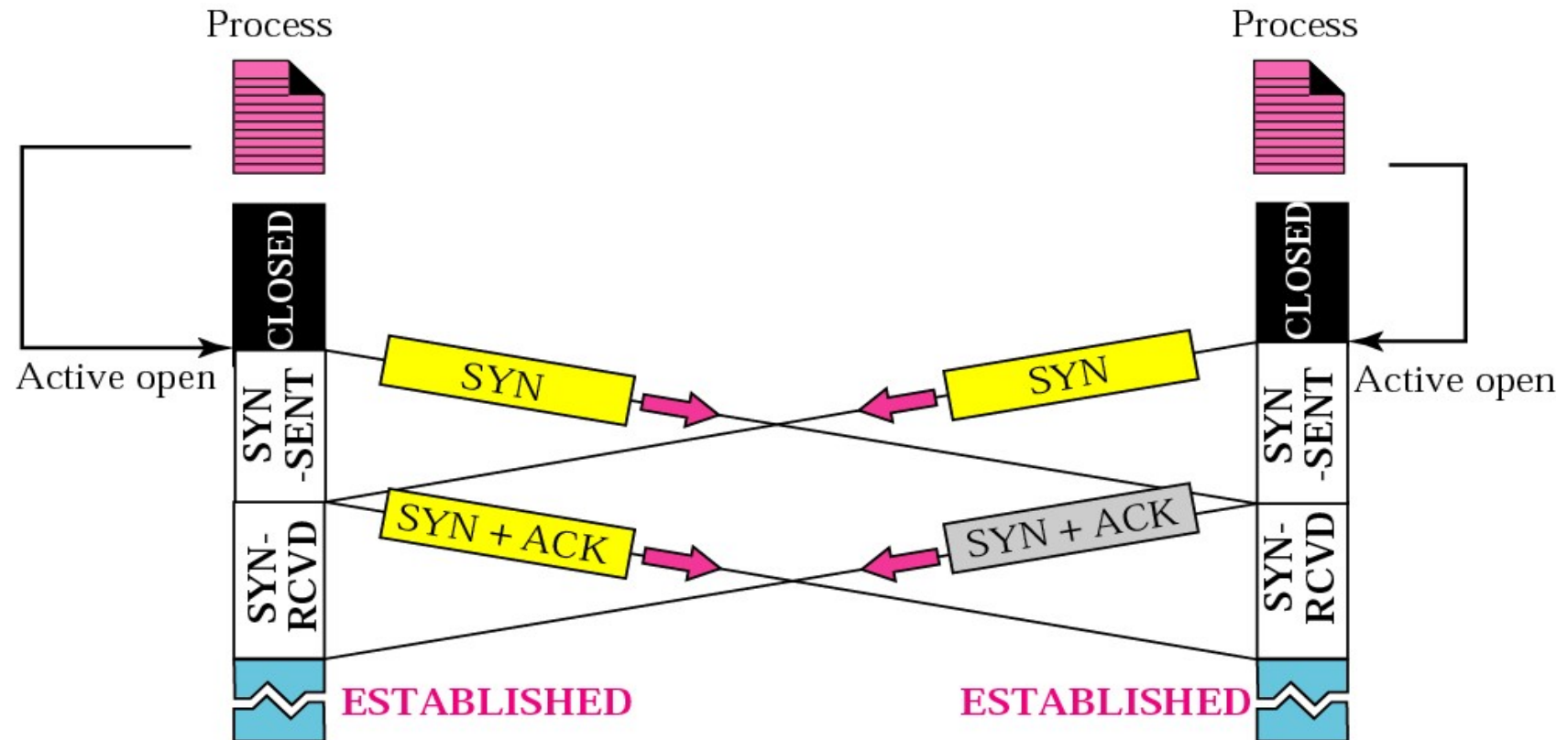
Cenário comum do TCP



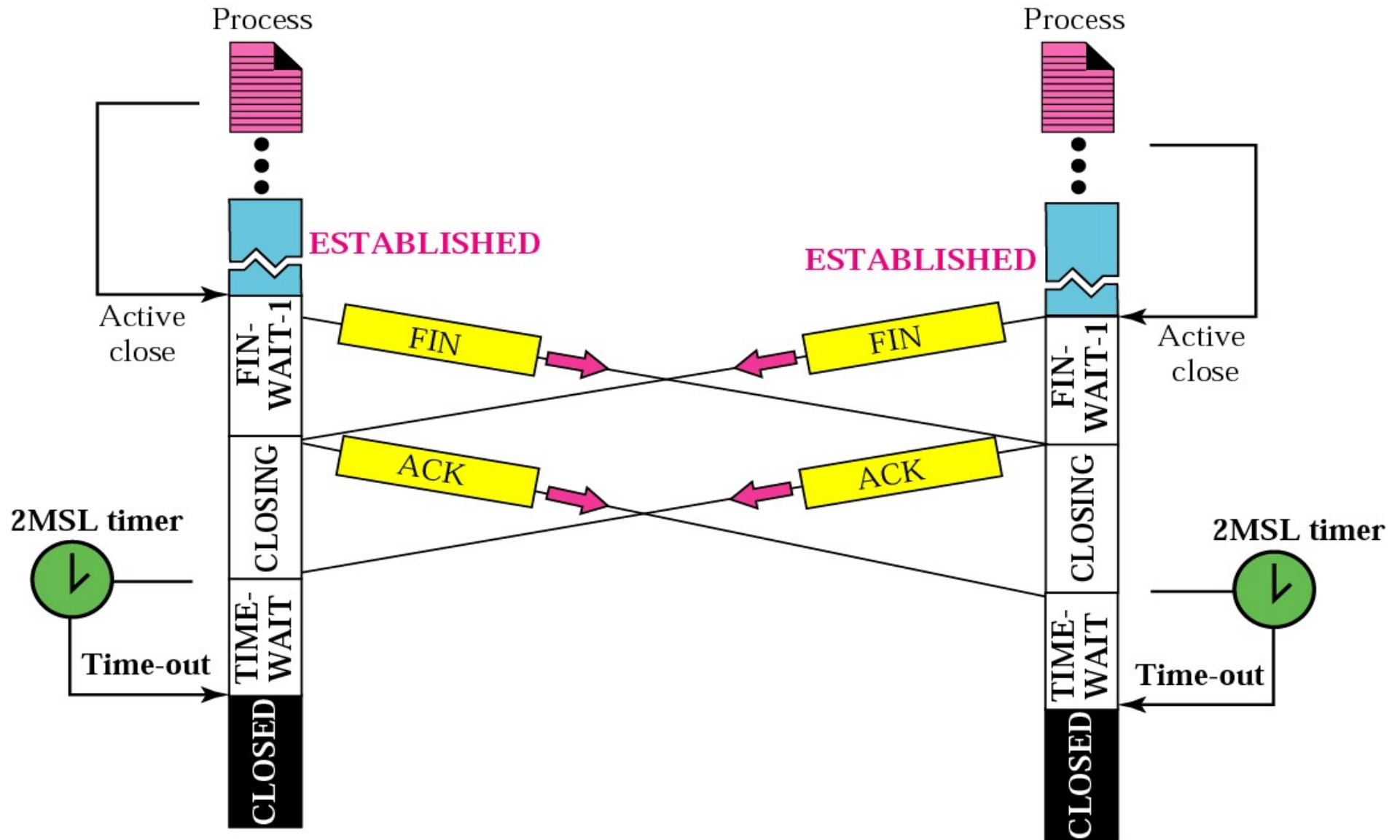
Handshake de 3 vias



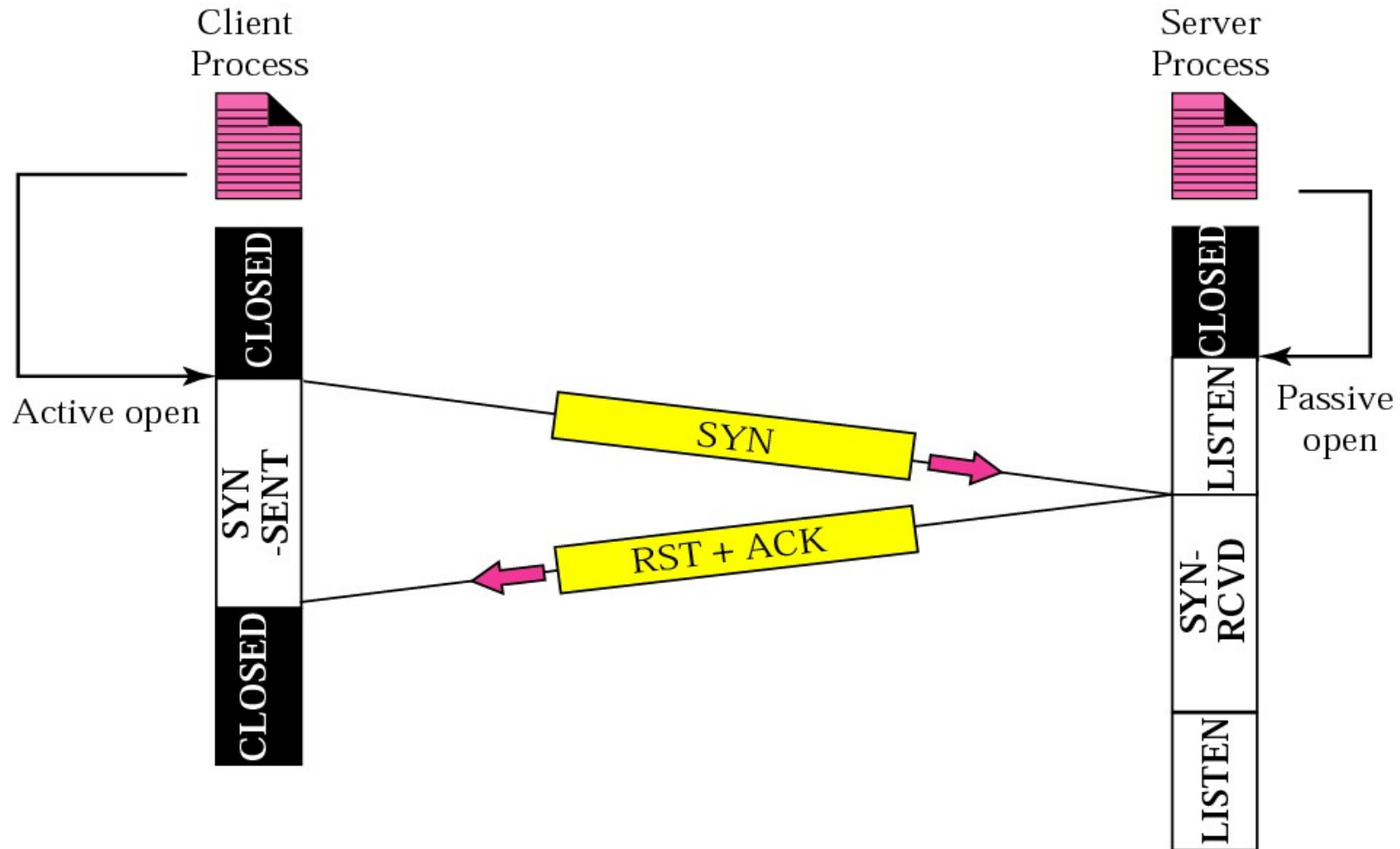
Abertura simultânea



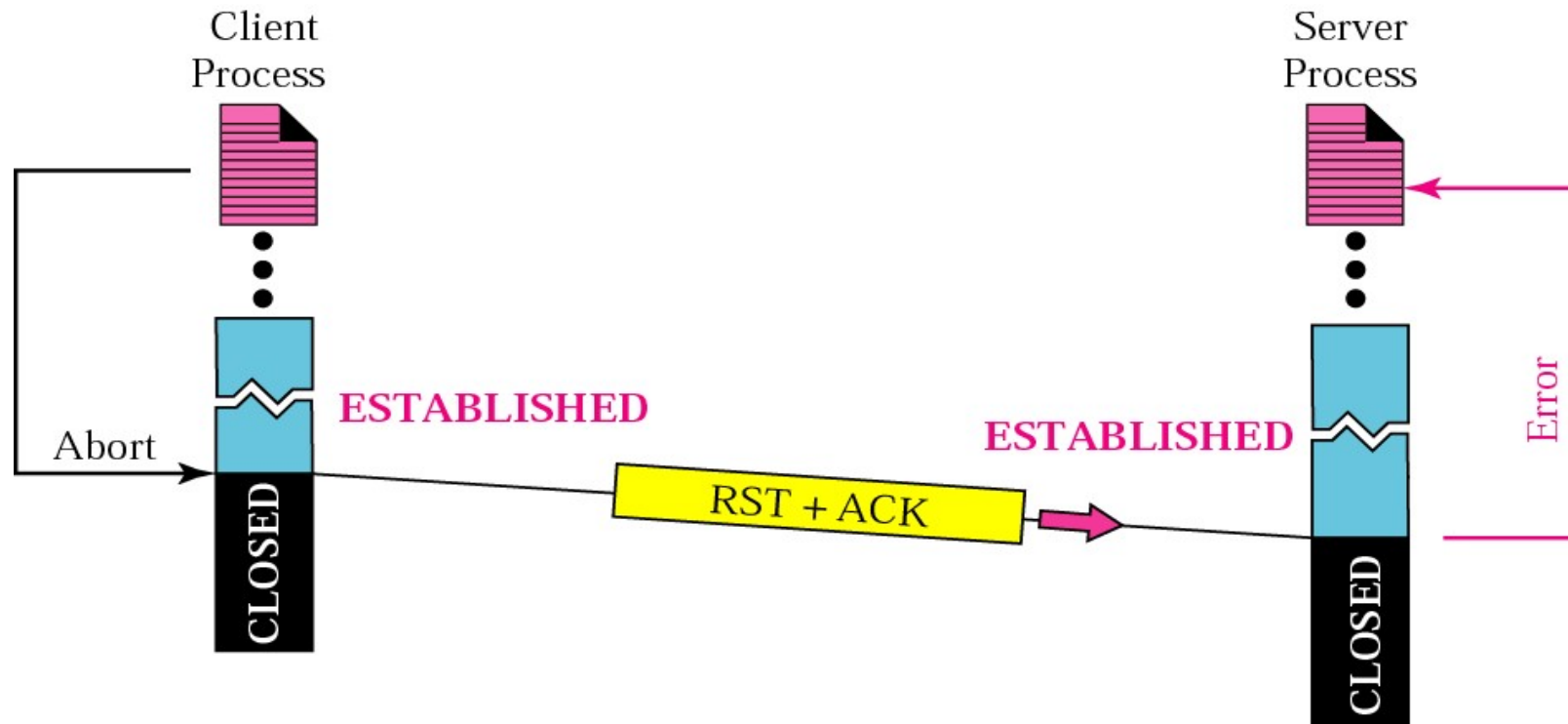
Encerramento simultâneo



Negando uma conexão



Abortando uma conexão



Controle de congestionamento

Entre todas as camadas é a que verdadeiramente precisa diminuir a taxa de transmissão de dados.

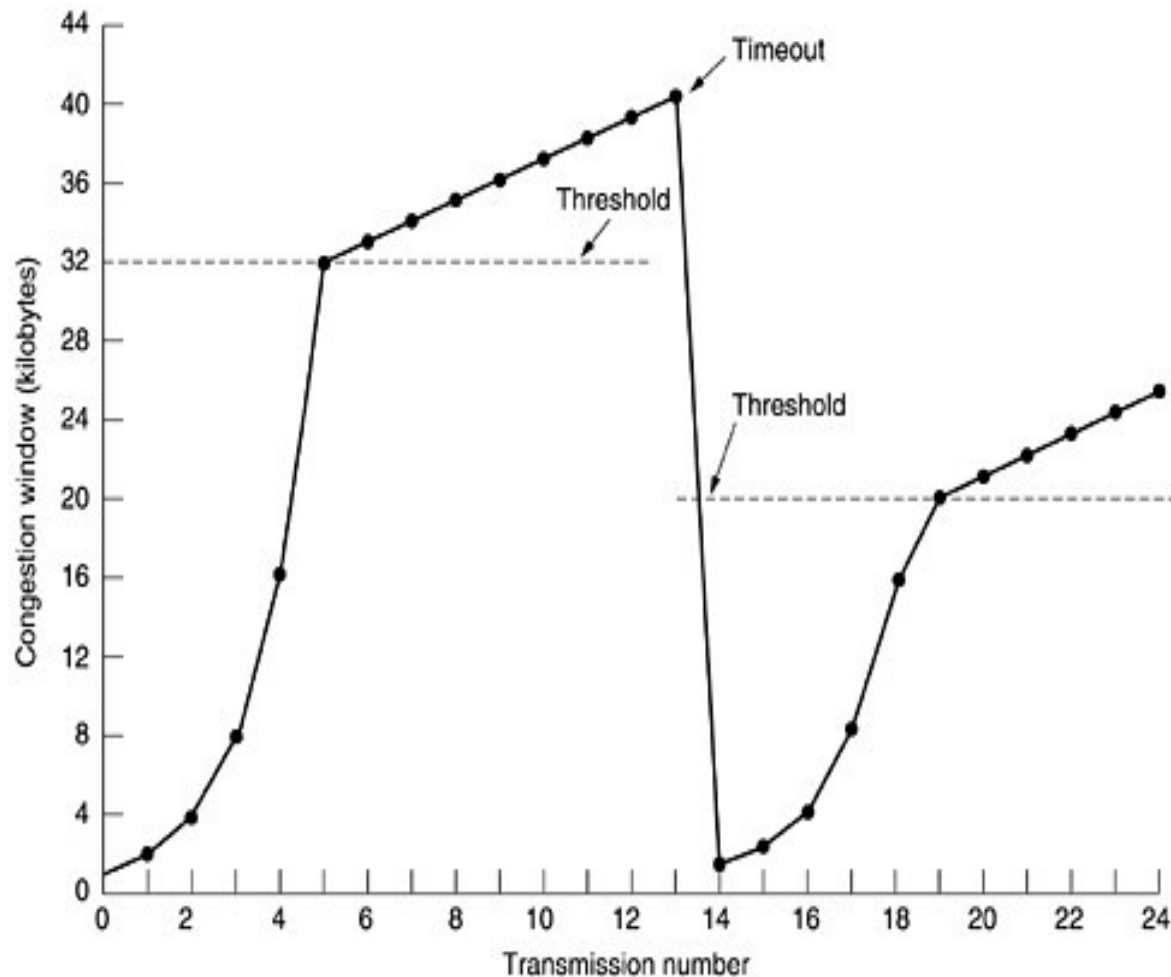
Uma solução é a manipulação do tamanho das janelas deslizantes dinamicamente.

Timeout pode ser um parâmetro que indica a existência de um congestionamento, apesar de também indicar perda de pacote, na maioria das vezes indica congestionamento.

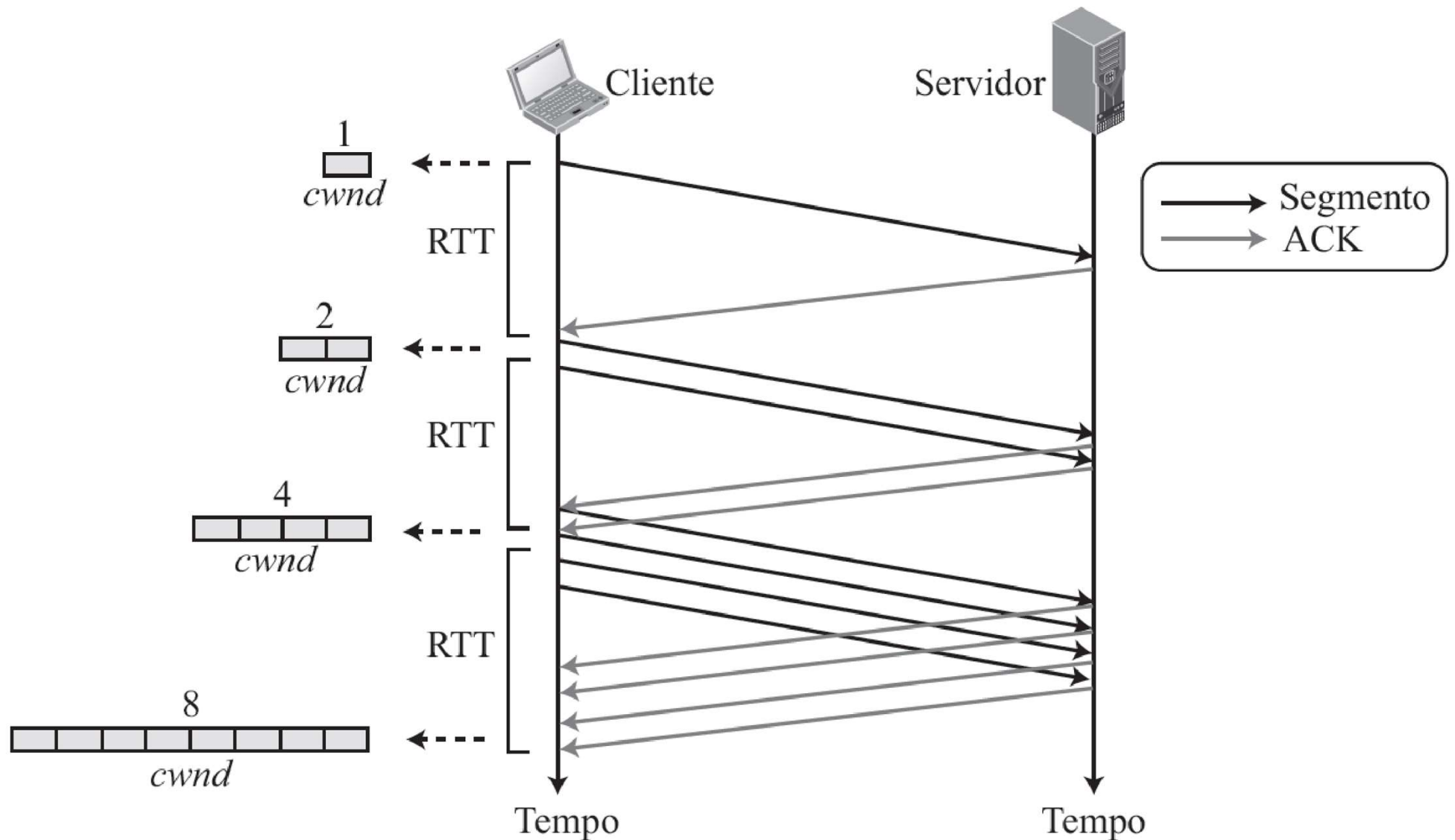
A capacidade da janela vai crescimento exponencialmente até que ocorra um timeout.

Controle de congestionamento

Na figura abaixo, onde o crescimento é exponencial até certo momento e depois reduz sua taxa de crescimento até o limite de 64 KB, ocorrendo um timeout antes de atingir o pico de 64 KB, o pico é reduzido e o tamanho mínimo da janela volta a número bem baixos e o algoritmo volta a seu início.

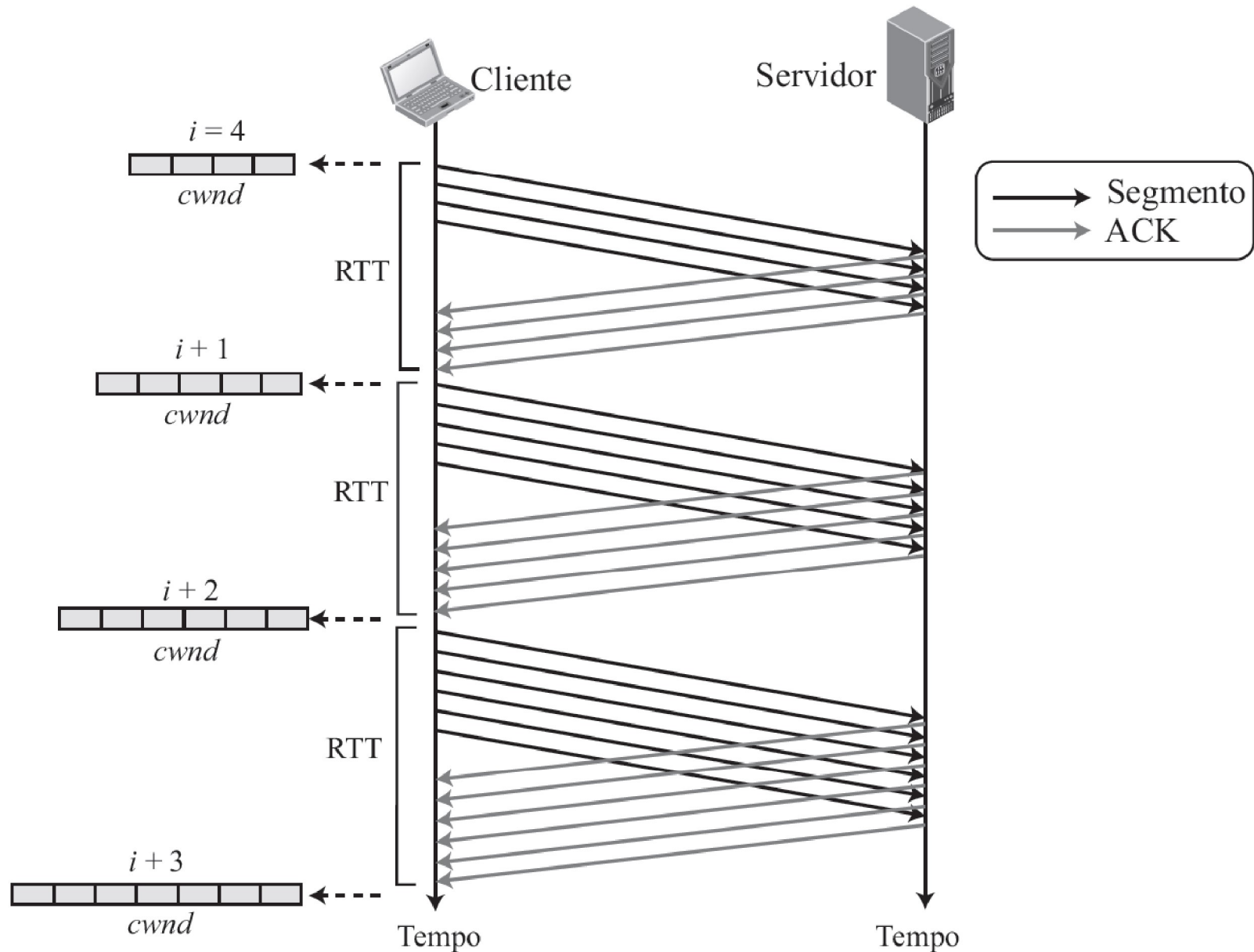


Partida lenta – Aumento exponencial

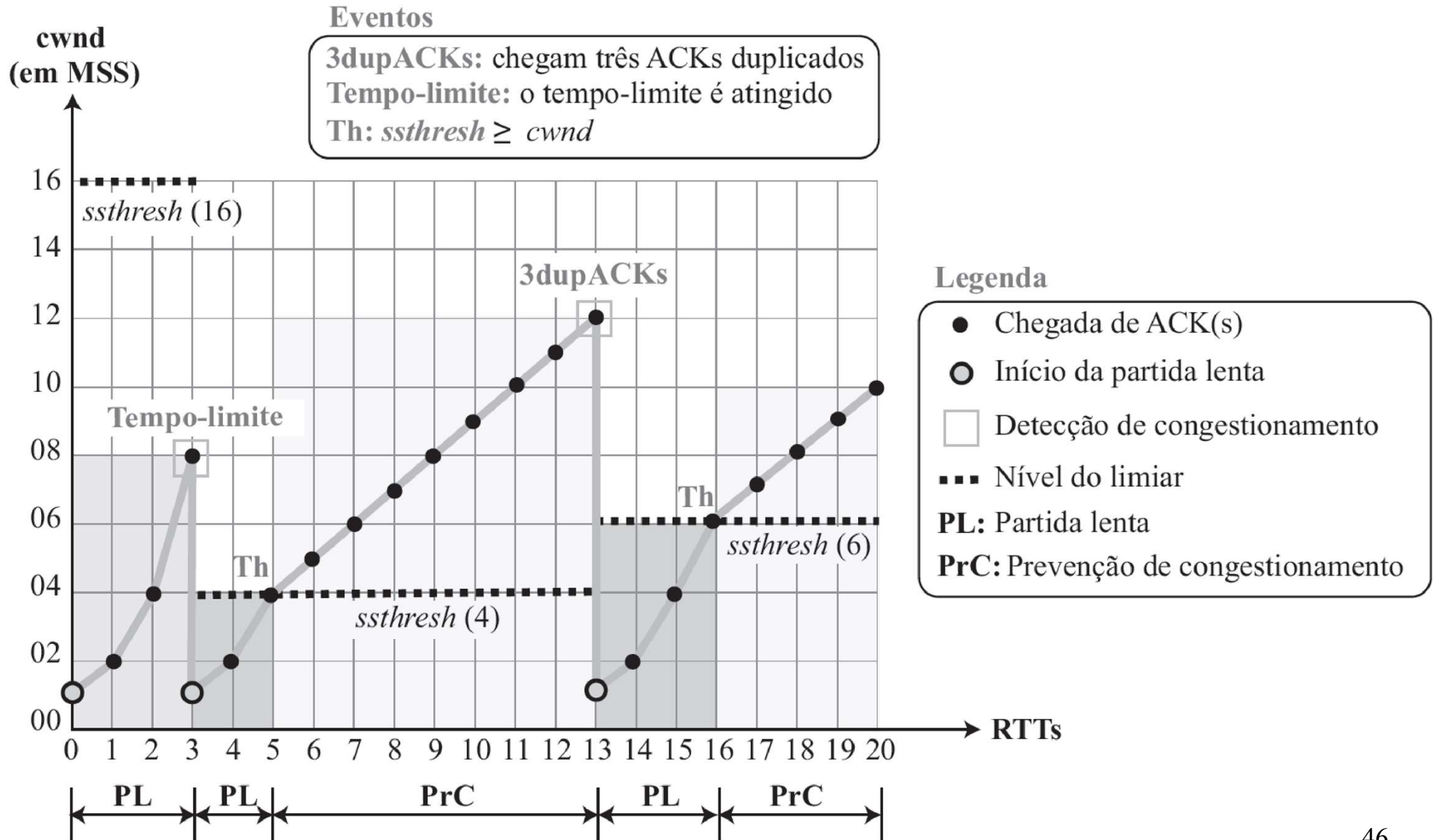


Prevenção de congestionamento

Aumento aditivo



TCP taho



TCP Reno

